ASSIGNMENT- 08

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Basic SQL Queries and Aggregate Functions

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

BASIC SQL QUERIES AND AGGREGATE FUNCTIONS

1. Introduction

SQL (Structured Query Language) is used to retrieve, filter, sort, and analyze data stored in a relational database. In an Online Food Delivery System, SQL queries help retrieve customer information, display restaurant details, analyze orders, and calculate sales statistics.

This assignment demonstrates basic SQL queries using the `FoodDeliveryDB` database created in the previous assignments. It includes the `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING` clauses, along with the aggregate functions `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`.

2. Objectives

- To retrieve data from database tables using SQL.
- To filter records using the `WHERE` clause.
- To sort records using the `ORDER BY` clause.
- To group records using the `GROUP BY` clause.
- To filter grouped results using the `HAVING` clause.
- To perform calculations using aggregate functions.
- To understand how SQL helps analyze food delivery operations.

3. Filtering Records Using WHERE

The `WHERE` clause filters records based on a specified condition.

Query 1: Display customers whose names are stored in the database

```
USE FoodDeliveryDB;


SELECT customer_id, full_name, email

FROM Customers

WHERE customer_id <= 3;
```

Sample Output:

customer_id	full_name	email
1	Mohammed Sinan	sinan@example.com
2	Rahul Menon	rahul@example.com
3	Ananya Nair	ananya@example.com

Explanation: This query displays customers whose IDs are less than or equal to 3.

Query 2: Display orders with a total amount greater than ₹250

```
SELECT order_id, customer_id, total_amount, order_status

FROM Orders

WHERE total_amount > 250;
```

Sample Output:

order_id	customer_id	total_amount	order_status
1	1	320.00	Delivered
3	3	370.00	Delivered
4	4	360.00	Out for Delivery

Explanation: This query retrieves orders whose total amount exceeds ₹250.

Query 3: Display restaurants with a rating of 4.5 or higher

```
SELECT restaurant_id, restaurant_name, rating
```

FROM Restaurants

WHERE rating >= 4.5;

Sample Output:

restaurant_id	restaurant_name	rating
2	Spice Garden	4.5
3	Pizza Corner	4.6

Explanation: This query filters restaurants based on their ratings.

4. Sorting Records Using ORDER BY

The `ORDER BY` clause sorts query results in ascending (`ASC`) or descending (`DESC`) order.

Query 4: Display restaurants in descending order of rating

SELECT restaurant_name, rating

FROM Restaurants

ORDER BY rating DESC;

Sample Output:

restaurant_name	rating
Pizza Corner	4.6
Spice Garden	4.5

Food Palace	4.3
Burger Hub	4.2
Sweet Treats	4.0

Explanation: The restaurants are arranged from the highest rating to the lowest rating.

Query 5: Display orders from highest to lowest total amount

SELECT order_id, customer_id, total_amount

FROM Orders

ORDER BY total_amount DESC;

Sample Output:

order_id	customer_id	total_amount
3	3	370.00
4	4	360.00
1	1	320.00
5	5	220.00
2	2	180.00

Explanation: This query sorts orders by their total amount in descending order.

5. Grouping Records Using GROUP BY

The `GROUP BY` clause groups records that have the same value in a specified column. It is commonly used with aggregate functions.

Query 6: Count the number of orders for each order status

```
SELECT order_status, COUNT(*) AS total_orders
FROM Orders
GROUP BY order_status;
```

Sample Output:

order_status	total_orders
Delivered	3
Out for Delivery	1
Preparing	1

Explanation: This query groups orders by their status and counts the orders in each group.

Query 7: Calculate total sales for each restaurant

```
SELECT r.restaurant_name,
    SUM(o.total_amount) AS total_sales
FROM Restaurants r
JOIN Orders o
  ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name;
```

Sample Output:

restaurant_name	total_sales
Food Palace	320.00
Spice Garden	180.00
Pizza Corner	370.00
Burger Hub	360.00
Sweet Treats	220.00

Explanation: This query joins the `Restaurants` and `Orders` tables and calculates the total order amount associated with each restaurant. The figures represent the sample order totals, not necessarily completed or paid sales.

6. Filtering Grouped Results Using HAVING

The `HAVING` clause filters groups after the `GROUP BY` operation. Unlike `WHERE`, which filters individual records before grouping, `HAVING` can filter results using aggregate calculations.

Query 8: Display restaurants with total order amounts greater than ₹300

```
SELECT r.restaurant_name,
    SUM(o.total_amount) AS total_sales
FROM Restaurants r
JOIN Orders o
  ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
HAVING SUM(o.total_amount) > 300;
```

Sample Output:

restaurant_name	total_sales
Food Palace	320.00
Pizza Corner	370.00
Burger Hub	360.00

Explanation: This query displays only restaurants whose combined order amounts exceed ₹300.

Query 9: Display order statuses with more than one order

```
SELECT order_status, COUNT(*) AS total_orders
FROM Orders
GROUP BY order_status
HAVING COUNT(*) > 1;
```

Sample Output:

order_status	total_orders
Delivered	3

Explanation: This query groups orders by status and displays only those statuses associated with more than one order.

7. Aggregate Functions

Aggregate functions perform calculations on a set of records and return a single result for each group or for the entire result set.

Query 10: COUNT() – Count the total number of customers

```
SELECT COUNT(*) AS total_customers
FROM Customers;
```

Sample Output:

total_customers
5

Explanation: `COUNT()` counts the number of records in the `Customers` table. The sample database contains five customers.

Query 11: SUM() – Calculate the total value of all orders

```
SELECT SUM(total_amount) AS total_order_value
FROM Orders;
```

Sample Output:

total_order_value
1450.00

Explanation: `SUM()` adds the `total_amount` values of all five sample orders.

Calculation:

₹320 + ₹180 + ₹370 + ₹360 + ₹220 = **₹1,450**

Query 12: AVG() – Calculate the average order amount

```
SELECT AVG(total_amount) AS average_order_amount
FROM Orders;
```

Sample Output:

average_order_amount
290.000000

Explanation: `AVG()` calculates the average of all order amounts.

Calculation:

₹1,450 ÷ 5 = **₹290**

The number of decimal places displayed may vary depending on the SQL client.

Query 13: MIN() – Find the minimum order amount

```
SELECT MIN(total_amount) AS minimum_order_amount
FROM Orders;
```

Sample Output:

minimum_order_amount
180.00

Explanation: `MIN()` returns the smallest order amount in the `Orders` table.

Query 14: MAX() – Find the maximum order amount

```
SELECT MAX(total_amount) AS maximum_order_amount
FROM Orders;
```

Sample Output:

maximum_order_amount
370.00

Explanation: `MAX()` returns the largest order amount in the `Orders` table.

8. Combined Query Using Multiple SQL Clauses

Query 15: Find order statuses with an average order amount greater than ₹250

```
SELECT order_status,
    COUNT(*) AS total_orders,
    AVG(total_amount) AS average_amount
FROM Orders
GROUP BY order_status
HAVING AVG(total_amount) > 250
ORDER BY average_amount DESC;
```

Sample Output:

order_status	total_orders	average_amount
Delivered	3	290.000000
Out for Delivery	1	360.000000

Explanation: This query groups orders by status, calculates the number of orders and average order amount for each group, filters the groups using `HAVING`, and sorts the results using `ORDER BY`.

Note: The correct output for this query is ordered by average amount in descending order. Therefore, the rows should appear as follows:

order_status	total_orders	average_amount
Out for Delivery	1	360.000000
Delivered	3	290.000000

9. Summary of SQL Clauses and Functions

SQL Clause/Function	Purpose	Example
`WHERE`	Filters individual records	`WHERE total_amount > 250`
`ORDER BY`	Sorts query results	`ORDER BY rating DESC`
`GROUP BY`	Groups records with common values	`GROUP BY order_status`
`HAVING`	Filters grouped results	`HAVING COUNT(*) > 1`
`COUNT()`	Counts records	`COUNT(*)`
`SUM()`	Calculates the total	`SUM(total_amount)`

`AVG()`	Calculates the average	`AVG(total_amount)`
`MIN()`	Finds the minimum value	`MIN(total_amount)`
`MAX()`	Finds the maximum value	`MAX(total_amount)`

10. Result

The SQL queries were designed to retrieve, filter, sort, and analyze data from the Online Food Delivery System database. The `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING` clauses demonstrated record filtering, sorting, grouping, and group-level filtering. The aggregate functions `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()` demonstrated how to calculate useful statistics from database records.

11. Conclusion

This assignment demonstrates the practical application of basic SQL queries and aggregate functions in a relational database. These operations help a food delivery platform analyze customers, restaurants, orders, and order values. Understanding these SQL concepts is essential for retrieving meaningful information and making data-driven decisions.